

CivicSync

A-to-Z Complete Project Architecture, Data, UI, Agent Split & Build Specification

SIH 2026 | PS 26043 | Government of Jharkhand | Smart Education | Software | Mind Mavericks

Purpose: Build CivicSync with three agents and merge through Git. This document defines the product architecture, UI, data, APIs, AI, security, testing, demo and deployment rules.

Locked product flow: Report -> Verify -> Understand -> Cluster/Prioritize -> Match -> Solve -> Validate -> Implement -> Measure.

1. Final Product Definition

- **CivicSync converts citizen-originated societal problems into structured, prioritized challenges; connects them to relevant capabilities; supports solution development and human validation; creates an implementation pathway; and measures outcomes with evidence.**
- **Smart Education connection:** real community problems become research, project and experiential-learning opportunities for students, faculty and universities, with industry participation for deployment.

2. Frozen System Architecture

Browser / Users | React + Vite + Tailwind | REST / JSON | Node.js + Express | | +-----+ v | PostgreSQL v ^ Python + FastAPI | | +---- persisted AI results ----+ | Rule-Based NLP Pattern Analysis

- **Rules:** frontend never accesses PostgreSQL; Node/Express owns application workflow; FastAPI exposes focused AI endpoints; AI results are persisted with confidence and processing metadata.

3. Three-Agent Ownership

- **Agent 1 - Frontend/UI:** owns only /frontend. Builds public and role-based screens, components, navigation, state handling and API consumption. No DB schema, backend business logic or AI algorithm edits.
- **Agent 2 - Database + Backend + FastAPI integration:** owns /database and /backend. Builds bootstrap schema, migrations, seeds, Node/Express APIs, auth, RBAC, orchestration, persistence and Node-to-FastAPI integration.
- **Agent 3 - AI/NLP:** owns /ai-service. Builds problem intelligence, expertise extraction, duplicate/related detection, priority logic and AI-assisted solution evaluation.
- **Shared contract files:** docs/api.md, docs/ai-contract.md, docs/database.md, docs/architecture.md, README.md, .env.example.

4. Repository Structure

civicsync/ frontend/ # Agent 1 backend/ # Agent 2 ai-service/ # Agent 3 database/ # Agent 2 docs/ # shared contracts .gitignore README.md

5. Global UI System

- Style: modern civic-tech + education + innovation. White/green base, restrained secondary accents, clean cards, status badges, meaningful charts, responsive layouts.
- Global shell: top navigation for logo/search/notifications/profile; role-based sidebar; responsive content area.
- Shared components: buttons, forms, cards, badges, tabs, tables, timelines, modals, toasts, empty/loading/error states.
- Never hard-code success data in production UI; use API response states.

6. Public Screens

- Landing: value proposition, Report a Problem, Explore Challenges, how it works, stakeholder roles.
- Challenge Explorer: search + filters for domain, severity, location, priority and status.
- Challenge Detail: problem, location, evidence, community support, priority, expertise, related reports and lifecycle.
- Login / Register: validation and role-aware access.

7. Citizen Screens

- Dashboard: My Problems, Under Review, In Progress, Resolved, notifications.

- Report Problem: title, description, location, category hint, affected people, urgency, evidence, consent/public visibility.
- AI Analysis: domain, subdomain, problem type, severity, urgency, required expertise, summary, confidence.
- Related Problems: duplicate/related results, cluster and master challenge.
- Problem Tracking: lifecycle timeline and updates.

8. Student / Researcher Screens

- Dashboard: recommended challenges, joined challenges, submitted solutions, selected projects.
- Challenge Workspace: problem context, required expertise, existing team, resources, join challenge.
- Solution submission: title, summary, technology/approach, cost, duration, beneficiaries, infrastructure, scalability, risks and evidence.
- Solution Detail: evaluation, comments, status and implementation progression.

9. Faculty / Expert Screens

- Dashboard: expertise matches, review requests and challenge participation.
- Profile: expertise, research topics, projects, facilities, location, availability/capacity and prior work.
- Review: technical/contextual assessment, comments, approve/request changes/reject.

10. University Screens

- Dashboard: challenges, teams, expertise, facilities, projects and impact.
- Capability profile: departments, faculty, labs/facilities, research areas and previous projects.
- Actions: nominate experts, join relevant challenges, participate in teams and review/impact workflows.

11. Startup / MSME Screens

- Implementation Opportunities: challenge, selected solution, requirements, location, timeline and status.
- Partner Profile: technical capability, sector, experience, performance, rating, capacity and deployment ability.
- Partner Comparison: configurable capability/experience/performance/cost/timeline score.
- Actions: Express Interest -> selection -> deployment updates -> outcome evidence.

12. Authority / Moderator Screens

- Dashboard: new/high-priority/flagged problems, clusters, projects and impact.
- Verification: inspect evidence, AI output, related reports and priority signals.
- Actions: verify, flag, moderate, merge into master challenge, assign reviewer, update status.
- Review implementation/impact records; final critical decisions remain human-controlled.

13. Admin Screens

- Users, roles, organizations, expertise taxonomy, moderation, analytics, audit logs, configuration and system health.
- Admin governance must be separate from ordinary role workflows.

14. Database - Core Tables

users user_verifications organizations profiles expertise capabilities problems problem_analysis problem_evidence problem_votes problem_comments problem_relations problem_clusters problem_expertise priority_scores matches teams team_members collaborations solutions solution_evaluations solution_reviews implementation_opportunities implementation_partners partner_scores impact_plans impact_metrics impact_evidence impact_verifications impact_scores reputation impact_points badges notifications audit_logs

- **Core relationship:** User -> Problem -> AI Analysis -> Expertise -> Match -> Team -> Solution -> Review -> Implementation -> Impact -> Evidence -> Reputation.

15. Database Rules

- Use primary/foreign keys consistently; add created_at/updated_at to lifecycle entities.
- Keep public/restricted citizen fields explicit and enforce visibility server-side.

- Do not store Aadhaar in MVP.
- Store AI service/version/confidence with AI outputs.
- Use transactions for state changes such as verification, selection and lifecycle transitions.

16. Important Current Database Asset Audit

- The supplied database ZIP contains **18 migration files** but **schema.sql** and **seed.sql** are **0 bytes**. Therefore a clean empty-database bootstrap must be created before merge.
- Two migration files share version prefix **017**: `017_trust_and_anti_gaming.sql` and `017_verification_layer.sql`. Rename to unique ordered versions before applying migrations.
- Some migrations alter tables that must already exist. Build a deterministic base schema first, then ordered migrations, then seed data.
- CI must prove that the full database can be created from empty PostgreSQL.

17. Backend Responsibilities

- Auth: register/login/JWT/verification state; password hashing with bcrypt.
- RBAC: server-side role checks on every protected resource.
- Problem lifecycle: CRUD, verification, evidence metadata, status and public/restricted fields.
- Orchestration: FastAPI calls, AI persistence, clustering/priority/matching services, solutions, implementation, impact and admin.
- Auditability: reviewer actions, critical state changes and AI processing metadata.

18. API Contract

POST /api/auth/register POST /api/auth/login POST /api/auth/verify-otp POST /api/problems GET /api/problems GET /api/problems/:id PATCH /api/problems/:id
 POST /api/ai/analyze POST /api/ai/cluster POST /api/ai/priority POST /api/problems/:id/vote POST /api/problems/:id/comments GET /api/expertise POST
 /api/expertise POST /api/matches/run GET /api/problems/:id/matches POST /api/collaborations POST /api/teams PATCH /api/teams/:id POST
 /api/problems/:id/solutions GET /api/problems/:id/solutions POST /api/solutions/:id/evaluate POST /api/solutions/:id/reviews POST
 /api/solutions/:id/implementation POST /api/implementation/:id/impact-plan POST /api/impact/:id/metrics POST /api/impact/:id/evidence

- Exact route names may evolve, but domain boundaries and payload contracts must remain stable once frozen.

19. AI/NLP - MVP

- FastAPI /analyze receives title, description, location, optional category hint, media metadata, started_at, affected_people and urgency.
- Output: domain, subdomain, problem_type, severity, urgency, entities, required_expertise, summary, confidence, evidence_quality.
- Current MVP uses **rule-based NLP + pattern analysis**. Do not claim trained ML/deep learning/LLM unless actually implemented and tested.
- AI does not prove medical/legal/engineering causes and does not make final governance/procurement decisions.

20. Duplicate / Related Detection

- MVP signals: normalized text overlap + category/subcategory + geographic proximity + time window.
- Return DUPLICATE / RELATED / UNRELATED plus reason and similarity.
- Human confirmation should be required where relationship changes a master challenge.

21. Priority Engine

- Priority = f(community support, severity, urgency, people affected, geographic spread, evidence quality, authorized signals).
- Community support is one signal, not the whole definition of importance.
- Expose component values so judges can see why priority changed.

22. Explainable Matching Engine

- Input: required expertise R and candidate capability set C.
- **MVP base score**: $|R \cap C| / |R| \times 100$.
- Return score + overlapping capabilities + explanation text.

- Future: add relevance, experience, location, facilities and availability.

23. Solution Evaluation

- Solution fields: title, summary, technology/approach, cost, duration, beneficiaries, infrastructure, scalability, risks and evidence.
- Example rubric: technical feasibility 25%, social impact 25%, cost effectiveness 15%, scalability 15%, sustainability 10%, innovation 10%.
- Rubric weights are configurable examples, not scientific truth. AI is advisory; human reviewer selects.

24. Implementation Layer

- Selected solution -> implementation opportunity -> partner interest -> partner comparison -> selection -> deployment.
- Example partner score: technical capability 30%, experience 20%, past performance 15%, rating 15%, cost 10%, timeline 10%.
- CivicSync facilitates opportunity/EOI; it does not replace statutory procurement.

25. Impact & Evidence

- Impact Plan: baseline, target, KPI, measurement method, period, evidence source, actual result.
- Evidence: photos, sensors/IoT, lab/test reports, government/institutional records, usage stats, field videos, GPS, community feedback.
- Verification levels: L1 self-reported, L2 community verified, L3 data/document verified, L4 independent/authorized.
- Impact scoring should use domain-specific KPIs; example factors can include outcome improvement, people/area benefited, evidence quality, sustainability, satisfaction and scalability.

26. Reputation / Rewards / Trust

- Reputation = verified contribution + quality + successful delivery + verified impact.
- Impact Points and badges are recognition, not a monetary core.
- MVP trust controls: OTP/account verification, rate limits, duplicate checks, self-vote prevention, moderation hooks and abuse penalties where proven.

27. Security & Privacy

- JWT, bcrypt, RBAC, input validation, file type/size limits, rate limiting, audit logs, environment-variable secrets and structured logging.
- Consent must distinguish public/anonymized/restricted publication and authorized contact/location access.
- No Aadhaar storage in MVP.

28. Supplied Data - Exact MVP Snapshot

- The supplied data ZIP contains **22 CSV datasets / 375 rows** and **2 JSON test files / 54 cases**.
- **AI regression:** 42 cases. **Matching:** 12. **Duplicate/related:** 24. **Priority:** 25.
- **Domain/capability assets:** 45 expertise records; 22 faculty; 38 faculty-expertise mappings; 10 institutions; 31 institution-expertise mappings; 12 facilities; 21 departments; 10 government departments; 8 industry; 8 innovation entities; 6 implementation partners.
- **Lifecycle/demo assets:** 30 challenges; 6 clusters; 12 solutions; 8 projects; 10 impact metrics; 10 evidence records; 15 users; 12 notifications; 12 reputation records.
- Use synthetic/demo labels honestly. Verified-real assets should be presented as verified-real only where the source data marks them so.

29. External Data - Verified Sources for Future Enrichment

- **AISHE:** official Ministry of Education higher-education data/portal; useful for institution discovery. <https://aishe.gov.in/>
- **UDISE+:** official school education MIS with school profile, infrastructure, student and teacher information; some data access is controlled. <https://www.udiseplus.gov.in/>
- **Jharkhand GIS:** official geospatial portal with Health, Agriculture, School, Industry and other layers. <https://gis.jharkhand.gov.in/>
- **Jal Jeevan Mission:** water-quality monitoring/surveillance resources. <https://jaljeevanmission.gov.in/>

- **CPCB:** National Water Quality Monitoring Programme/resources. <https://www.cpcb.nic.in/>
- **Soil Health Card:** official soil-health scheme and parameters. <https://soilhealth.dac.gov.in/>
- **DPIIT Startup OGD:** state/sector/year startup dataset. <https://data.gov.in/catalog/startup-recognized-dpiit/>
- **Rule:** external sources enrich the system; the hackathon demo must remain functional without them.

30. UI-to-API Integration Rules

- All frontend API calls go through a common service layer. Use VITE_API_URL; do not hard-code URLs in components.
- Centralize JWT attachment/401/403 handling.
- UI must reflect backend state; never fake a successful API result for the demo unless clearly marked as mock/prototype data.
- Use role-based navigation, but enforce authorization again on the server.

31. Git Branches & Ownership

- **main** - integrated stable branch.
- **feature/frontend** - Agent 1 only.
- **feature/backend-db** - Agent 2 only.
- **feature/ai** - Agent 3 only.
- Before merge: lint, unit tests, relevant API/AI tests, build and smoke test.
- Integration branch: combine and test all three before main.

32. Merge Order

- 1) Fix database bootstrap + migration numbering.
- 2) Freeze API/AI/database contracts.
- 3) Backend auth/RBAC/problem APIs.
- 4) AI service + regression tests.
- 5) FastAPI integration into backend.
- 6) Cluster/priority/matching.
- 7) Solution/review/implementation/impact.
- 8) Frontend P0 screens + API integration.
- 9) Full end-to-end test + demo hardening.

33. Final Demo Story

- Master scenario: citizen reports yellow handpump water and related health complaints.
- Demo: report -> AI structured challenge -> related reports -> priority -> required expertise -> explainable matches -> student/researcher team -> solution -> AI-assisted evaluation -> human review -> seeded MSME implementation -> baseline/target/actual + evidence.
- Demo runs from seeded/local data; no paid AI/SMS/government API is required.

34. Testing & Acceptance

- Auth works; invalid access blocked; role restrictions enforced.
- Problem creation and validation work; public/restricted fields respected.
- AI returns valid schema and passes 42 regression cases.
- Duplicate/related logic passes 24 tests; priority passes 25 cases; matching passes 12 cases.
- Why-match is visible and deterministic for known seed cases.
- Solutions can be submitted; reviewers can approve/request changes/reject.
- Implementation partner scoring is deterministic.
- Impact data/evidence persists and displays.

- Full demo path works after fresh database initialization.

35. Deployment

- Frontend: Vercel/Netlify/static hosting with VITE_API_URL.
- Backend: Render/Railway/AWS small instance.
- AI: Render/Railway or same VM initially.
- Database: managed PostgreSQL.
- Storage: local demo/object storage later.
- Monitoring: basic logs and error logging. Secrets stay in environment variables.

36. Honest Scope Labels

- **IMPLEMENTED:** working in code.
- **PROTOTYPE:** simplified working demo.
- **PROPOSED:** designed but not built.
- **FUTURE:** later roadmap.
- Do not claim trained ML/deep learning, national expert registry, real field impact from seed values, official government integration, automated root-cause certainty or replacement of procurement.

37. External Research / Data Source Checks

- AISHE official portal currently describes annual higher-education survey coverage and institution data. <https://aishe.gov.in/>
- UDISE+ currently identifies itself as the Ministry of Education school MIS and provides school profile/facility/student/teacher information. <https://www.udiseplus.gov.in/>
- Jharkhand GIS currently lists Health, Agriculture, School, Industry and district geospatial portals. <https://gis.jharkhand.gov.in/>
- Jal Jeevan Mission currently publishes drinking-water quality monitoring/surveillance resources. <https://jaljeevanmission.gov.in/>
- CPCB currently publishes National Water Quality Monitoring Programme resources and data. <https://www.cpcb.nic.in/>
- DPIIT Startup OGD provides startup counts by year, sector and state. <https://data.gov.in/catalog/startup-recognized-dpiit>

38. Final Agent Master Instruction

- **Do not build CivicSync as a generic admin template.** Build the defined civic-tech workflow and use the supplied data.
- Do not regenerate or overwrite another agent's work. Do not invent endpoints, table names or payload shapes.
- Work phase-by-phase; after each phase run tests and report files changed, commands run, test results and known issues.
- Any architecture change requires an explicit update to the shared documentation and all affected contracts.
- The demo must work with local seeded data and must not depend on external services for its critical path.

39. Final Definition of Done

- A module is done only when persistence, API, validation, authorization, UI state handling, tests and demo behavior all work.
- Visual completion alone is not done.
- Before final submission: fresh DB test, API smoke test, AI regression, role-security test, end-to-end demo rehearsal and deployment smoke test.

Appendix - Verified Web References Used in the Blueprint

AISHE - Government of India, Ministry of Education

<https://aishe.gov.in/>

AISHE Higher Education Institution Directory

<https://dashboard.aishe.gov.in/>

UDISE+ - Ministry of Education

<https://www.udiseplus.gov.in/>

UDISE+ APIs

<https://api.udiseplus.gov.in/>

Jharkhand Geospatial Portal

<https://gis.jharkhand.gov.in/>

Jal Jeevan Mission - Water Quality

<https://jaljeevanmission.gov.in/>

CPCB - National Water Quality Monitoring Programme

https://www.cpcb.nic.in/uploads/WQM_Objective.pdf

Soil Health Card

<https://soilhealth.dac.gov.in/>

DPIIT Recognized Startups - OGD

<https://data.gov.in/catalog/startup-recognized-dpiit>

CPGRAMS

<https://pgportal.gov.in/>

MyGov

<https://www.mygov.in/>

Atal Innovation Mission

<https://aim.gov.in/>

Principal Scientific Adviser - Manthan

<https://www.psa.gov.in/manthan>

Verification note: external-source claims in this document were checked against current official pages in September 2026. Availability, access permissions, update frequency and APIs may change; use adapters and source metadata instead of hard-coding assumptions.